

YaySchedule - Technical Documentation

System Architecture, Data Sync Protocol & NLP Engine Details

1. Core Technology Stack

- Frontend Framework: Next.js (App Router, Turbopack, React 19) for fast server-side optimization and static prerendering.
- Local Database Layer: Dexie.js (wrapper for IndexedDB) providing full transactional support, schema migrations, and queries in-browser.
- Cloud Synchronization: Supabase (PostgreSQL client) with realtime triggers and structured sync logs.
- Text Editor & Calendar: TippiTap rich text framework and FullCalendar core wrapper.

2. The Local-First Sync Safeguard

To solve the data loss issue when adding a new/empty device, the client sync logic in 'lib/db/sync.ts' blocks automatic pushes if the device has never performed an initial pull. It checks 'last_sync_pull' in LocalStorage; if missing, it forces a pull and merges records locally using UUID comparison before initiating any server push.

3. Intent Classification & Offline NLP Pipeline

When a message is typed in the assistant chat:

1. Prefix Check: Checks if the input starts with '/note', '/reminder', or '/event'.
2. Provider Check: If the user has configured Gemini or OpenRouter, it calls the LLM with formatting prompts to correct spelling, polish grammar, and output JSON details.
3. Offline Fallback: If offline or keys are absent, it routes input to 'Chrono-node' (extracts dates/times) and 'Compromise' (normalizes and cleans titles), saving the structured item instantly in DexieDB.

4. Deployment & PWA Configuration

The project is deployed on Vercel with CNAME alias to 'www.yayschedule.com'. HTTPS is automatically managed via Let's Encrypt certificates. Service workers cached resources locally, allowing users to add the app to their mobile home screen with the custom-designed brand icon.